МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Национальный исследовательский университет ИТМО

НАУЧНО-ИССЛЕДОВАТЕЛЬСКАЯ РАБОТА

по направлению: «Математика и Информатика»

Тема: «Теория графов в решении прикладных задач: от классических алгоритмов обхода до метаэвристической оптимизации»

Выполнил:

Матвеев Дмитрий Михайлович,

учащийся 11А класса

МАОУ «Гимназия № 115» г. Уфы

Научный руководитель:

Петов Олег Владимирович, учитель информатики

Уфа — 2026

Содержание

1.Введение..... 3

1.1 Актуальность исследования..... 3

1.2 Цели и задачи..... 3

2.Теоретическая часть..... 4

2.1 История создания и практическая значимость..... 4

2.2 Что такое Граф, способы его представления и хранения..... 5

2.3 Основные алгоритмы на графах..... 6

2.4 Практическая задача и методы ее решения..... 7

3. Практическая часть..... 13

3.1 Демонстрация работы алгоритмов..... 13

3.2 Решение практической задачи и анализ полученных данных..... 14

4. Заключение..... 17

Список использованных источников и литературы..... 18

Приложение..... 19

АННОТАЦИЯ: Объем работы: 20 стр., **Количество рисунков:** 5, **Количество источников:** 9.

Ключевые слова: теория графов, алгоритмы обхода, задача коммивояжера(TSP), алгоритм Флойда-Уоршелла, DFS, BFS, муравьиный алгоритм(ACO), вычислительная сложность.

Объект исследования: взвешенные графы и алгоритмы на них.

Цель работы: анализ классических и метаэвристических алгоритмов на графах и сравнение их эффективности при решении задачи коммивояжера.

Результаты: Реализованы алгоритмы BFS, DFS, Флойда-Уоршелла, полного перебора, жадный и муравьиный алгоритмы. Проведен вычислительный эксперимент, доказывающий, что ACO обеспечивает баланс между скоростью O(n^2) и точностью, превосходя жадные методы на 20-30%.

1. Введение

1.1 Актуальность исследования:

Актуальность теории графов обусловлена тем, что она предоставляет инструменты для анализа сложных систем и структур, которые не всегда могут быть описаны стандартными методами. Теория графов позволяет моделировать и анализировать различные типы сетей.

Благодаря развитию информационных технологий и появлению больших объемов данных, теория графов становится все более востребованной в анализе и обработке информации. Она помогает выявлять закономерности, связи и структуры в данных и принимать обоснованные решения на их основе. Ее развитие и применение продолжают углубляться и расширяться, открывая новые возможности для анализа и оптимизации сложных систем и процессов.

1.2 Цели и задачи

Целью работы является:

1) Изучение теории Графов и ее практического применения.
2) Демонстрация работы различных алгоритмов на графы (на языке C++).
3) Решение поставленной практической задачи и проведение анализа.

Для достижения целей поставлены следующие задачи :

1) Написать основные понятия для понимания теории Графов.
2) Познакомиться с историей ее создания.
3) Рассмотреть что такое Граф, способы его представления и хранения.
4) Разобрать основные алгоритмы на графах и продемонстрировать их работу на конкретном примере.
5) Решить поставленную практическую задачу несколькими способами и провести анализ полученных данных.

2. Теоретическая часть

2.1 История создания и практическая значимость

Мой научно-исследовательский проект основан на Теории графов.

Исторически первым результатом в области теории графов является решение задачи о семи кёнигсбергских мостах, предложенное Л. Эйлером в 1736 году, выдающийся математик, член Петербургской академии наук, он считается родоначальником теории графов.

Задача заключалась в том, чтобы пройти по всем мостам Кёнигсберга так, чтобы не пересекать ни один из них дважды. Эйлер выяснил, что маршрут возможен только в том случае, если число нечётных вершин (вершин с нечётным числом рёбер) чётно. Если все вершины графа чётные, то можно начертить граф, не отрывая карандаш от бумаги, начиная и заканчивая в одной и той же вершине. При наличии более двух нечётных вершин такой маршрут невозможен. В графе кёнигсбергских мостов было четыре нечётные вершины, следовательно, пройти по всем мостам, не проходя дважды, невозможно.[1]

Графы в биологии.

Графы в биологии большую роль в биологический теории ветвящихся процессов. Для простоты мы рассмотрим только одну разновидность - размножение бактерий. Предположим, что через определенный промежуток времени каждая бактерия либо делится на две новые, либо погибает. Тогда для потомства одной бактерии мы получим бинарное дерево. Нас будет интересовать лишь один вопрос: в скольких случаях n - е поколение одной бактерии насчитывает ровно k потомков? Рекуррентное соотношение, обозначающее число необходимых случаев, известно в биологии под название процесса Гальтона - Ватсона. Его можно рассматривать как частный случай многих общих формул.[2]

Графы в теории массового обслуживания.

---

1 [электронный ресурс]: https://un-sci.com/ru/2019/06/20/istoriya-vozniknoveniya-teorii-grafov/

2 [литература]: Берж К. «Теория графов и её применения»

Понятие центральной вершины и центра графа появились в связи с задачами оптимального размещения пунктов массового обслуживания, таких как больницы, сберегательные банки, пожарные части, почтамты и т.п., когда важно минимизировать наибольшее расстояние от любой точки населенного пункта до ближайшего пункта.[3]

2.2 Что такое Граф, способы его представления и хранения

Определим базовые понятия для дальнейшего понимания сути моего проекта. Граф (V; E), где V - конечное множество, которое называется множеством вершин, а E - бинарное отношение между вершинами, которое называется множеством ребер. Граф как математический объект есть совокупность двух множеств — множества самих объектов (вершин) и множества их парных связей (ребер). По сути, графы помогают визуально представить всяческие сложные взаимодействия: пункты назначения и пути между ними, молекулы в веществе и так далее.[4]

Первый и самый распространенный способ - матрица смежности:

Это самый популярный и расточительный способ представления графа в памяти. Его уместно использовать, если количество рёбер велико.

Для хранения рёбер используется двумерная матрица размерности [V, V], каждый [a, b] элемент которой равен 1, если вершины a и b являются смежными и 0 в противном случае.

Сложность по памяти: O(V^2). Сложность перечисления всех рёбер: O(V^2).

Тоже не менее популярный способ - матрица инцидентности:

Это самый расточительный способ хранения графа, его уместно использовать, если количество рёбер невелико.

Для хранения используется двумерная матрица размера [V, E], в каждом

3 [электронный ресурс]: clck.ru/39kVao

4 [литература]: Харари Ф. «Теория графов»

столбце которой записано одно ребро таким образом: напротив вершин, инцидентных этому ребру, записаны 1, в остальных случаях 0.

Сложность по памяти: O(V x E). Сложность перечисления всех рёбер: O(V x E).

Сложность по памяти: O(V + E).

Самый легкий и удобный способ - векторы смежности:

Для записи вектора смежности используется двумерная матрица размером [V, S], где S - максимальная степень вершины в графе.

В каждой строчке a записаны номера вершин, смежных с a, после чего записаны нули (несуществующие номера вершин).

Сложность по памяти: O(V x S). Сложность перечисления всех рёбер: O(V x E).[5]

2.3 Основные алгоритмы на графах

Теперь мы переходим к основным алгоритмам: поиск в глубину, поиск в ширину, Алгоритм Флойда — Уоршелла.

Начнем с поиска в глубину(Depth-first search, DFS) - один из методов обхода графа. Суть поиска в глубину состоит в том, чтобы идти вниз или вглубь графа, насколько это возможно. Он основан на принципе рекурсивного вызова функции для каждой еще не посещенной смежной вершины. Это можно представить как движение по ветвям дерева, идущего от корня вниз, пока не будет достигнут лист, а затем возвращение к предыдущим узлам для поиска других ветвей. Для его реализации используется стек(std::stack<Node>), который в свою очередь подчиняется принципу LIFO (Last-In, First-Out).

Второй алгоритм тоже для обхода графа, обход в ширину(breadth-first search, BFS) - это алгоритм обхода графа, который посещает вершины по уровням, начиная с корневой вершины. В основе я взял очередь

5 [электронный ресурс]: https://habr.com/ru/companies/otus/articles/675730/

(std::queue<Node>), которая подчиняется правилу FIFO (First-In, First-Out).

Алгоритм Флойда-Уоршелла является классическим методом динамического программирования для нахождения кратчайших путей между всеми парами вершин во взвешенном графе. Он работает для графов с как положительными, так и отрицательными весами ребер. Ключевая идея алгоритма Флойда-Уоршелла заключена в следующей рекуррентной формуле:

d(i, j, k) = min(d(i, j, k-1), d(i, k, k-1) + d(k, j, k-1))

Эта формула определяет кратчайшее расстояние d(i, j, k) между вершинами i и j с использованием вершин от 1 до k как минимум из двух вариантов:

1. d(i, j, k-1)**:** Кратчайший путь между i и j, используя только вершины от 1 до k-1 в качестве промежуточных. Это расстояние уже вычислено на предыдущем шаге.
2. d(i, k, k-1) + d(k, j, k-1)**:** Кратчайший путь, проходящий через вершину k. Он состоит из двух частей:
    - **d(i, k, k-1)**: Кратчайший путь от i до k, используя только вершины от 1 до k-1.
    - **d(k, j, k-1)**: Кратчайший путь от k до j, используя только вершины от 1 до k-1.

Алгоритм выбирает меньшее из этих двух расстояний, что и дает кратчайший путь между i и j с учетом вершин до k.

2.4 Практическая задача и методы ее решения.

В качестве практической задачи рассматривается задача коммивояжёра (TSP от англ. travelling salesman problem) — одна из самых известных задач комбинаторной оптимизации, заключающаяся в поиске самого выгодного маршрута, проходящего через указанные города хотя бы по одному разу с последующим возвратом в исходный город. Является NP-полной, то есть нельзя решить линейно в предельных значениях.

Для ее решения я написал проект. В основе лежат 3 алгоритма:

1. Полный перебор(Brute_force).
2. Жадный алгоритм(Greedy_Algorithm).
3. Эвристический алгоритм(ACO).

Начнем разбираться как они реализованы. Первым в моем проекте реализован Brut_force (алгоритм реализован в качестве эталона для верификации):

Представление графа:

Граф представляется матрицей смежности matrix, где matrix[i][j] — вес ребра (расстояние) между вершиной i и вершиной j. number_of_nodes — количество вершин в графе.

Генерация перестановок:

Функция permutation генерирует все возможные перестановки вершин, исключая начальную вершину number_of_start_node. Это можно записать как:

Пусть V = {1, 2, ..., n} — множество вершин. Начальная вершина — v_0. Тогда функция permutation генерирует все возможные перестановки множества V \ {v_0}. Каждая перестановка представляет собой последовательность посещения городов. Количество таких перестановок равно (n-1)!.

Добавление начальной и конечной вершин:

Цикл for (auto& i : paths) добавляет начальную вершину v_0 в начало и конец каждой сгенерированной перестановки. Это создает замкнутый маршрут, который начинается и заканчивается в v_0.

Вычисление длины маршрута:

Вложенный цикл for (size_t j = 0; j + 1 < i.size(); ++j) вычисляет длину каждого замкнутого маршрута i путем суммирования весов ребер:

Длина маршрута = Σ w(v_j, v_{j+1}) для j = 0, 1, ..., n-2, где w(v_j, v_{j+1}) =

matrix[i[j]][i[j+1]] — вес ребра между вершинами v_j и v_{j+1} в текущем маршруте.

Поиск минимального маршрута:

Условие if (tmp < min and *i.begin() == number_of_start_node) сравнивает длину текущего маршрута tmp с минимальной длиной min, найденной до этого момента. Если tmp меньше min, то min обновляется, и min_path сохраняет текущий минимальный маршрут.

Далее идет Greedy Algorithm — это метод решения задач путем разбиения на подзадачи и нахождения локально-оптимального решения. В моем проекте он реализован так:

Инициализация:

- Граф задан матрицей смежности matrix, где matrix[i][j] — вес ребра (расстояние) между вершиной i и j.
- n — количество вершин в графе.
- visited — массив булевых значений, отмечающих посещенные вершины (0 — непосещенная, 1 — посещенная).
- number_of_start_node — начальная вершина.

Жадный выбор следующей вершины:

Алгоритм итеративно выбирает следующую вершину, ближайшую к текущей посещенной вершине. На каждой итерации цикла while:

- tmp хранит индекс текущей посещенной вершины.
- Алгоритм перебирает все непосещенные вершины j (visited[j] == 0).
- min_distance_to_near хранит минимальное расстояние до ближайшей непосещенной вершины.
- Если расстояние от tmp до j (matrix[tmp][j]) меньше min_distance_to_near, то min_distance_to_near обновляется, и next устанавливается в j (индекс

ближайшей вершины).

Это можно описать формулой:

next = argmin_{j∈V, visited[j]=0} {matrix[tmp][j]}

где argmin — функция, возвращающая индекс элемента с минимальным значением.

Обновление пути и расстояния:

- Текущая вершина tmp помечается как посещенная (visited[tmp] = 1).
- tmp добавляется в min_path (последовательность посещенных вершин).
- Минимальное расстояние min увеличивается на расстояние до следующей вершины (min += matrix[next][tmp]).

Завершение цикла:

Цикл while повторяется до тех пор, пока все вершины не будут посещены (k < number_of_nodes).

Возвращение в начальную вершину:

После посещения всех вершин, алгоритм добавляет начальную вершину number_of_start_node в конец min_path, и min увеличивается на расстояние от последней посещенной вершины до начальной (min += matrix[tmp] [number_of_start_node]).

Третьим алгоритмом, применяемым для решения задачи коммивояжера в данной работе, является метод муравьиных колоний (Ant Colony Optimization, ACO). Данный алгоритм относится к классу роевого интеллекта и метаэвристических подходов, вдохновленный поведением муравьев при поиске пищи.[6] Муравьи оставляют феромоновый след на своем пути, и другие муравьи с большей вероятностью следуют за более сильными следами. ACO использует эту идею для поиска хороших решений в задачах оптимизации.

6 [электронный ресурс]: clck.ru/3Ew7z2

Феромоны: Каждое ребро (i, j) графа имеет связанное с ним количество феромона τ_{ij}. Чем больше феромонов на ребре, тем вероятнее муравьи выберут этот переход.

- **Вероятность перехода:** Каждый муравей k строит решение, последовательно выбирая ребра. Вероятность выбора муравьем k ребра (i, j) на шаге t определяется формулой:

  $p_{ij}^k(t) = [\tau_{ij}(t)]^\alpha * [\eta_{ij}]^\beta / \Sigma_{l \in J_i} [\tau_{il}(t)]^\alpha * [\eta_{il}]^\beta$

  где:

- $\tau_{ij}(t)$ — количество феромонов на ребре (i, j) в момент времени t.
- η_{ij} — эвристический параметр, отражающий желательность перехода, обратное расстояние между городами: $\eta_{ij} = 1/d_{ij}$, где d_{ij} — расстояние.
- α — параметр, управляющий влиянием феромонов. Большее значение α означает большее влияние феромонов (1.0).
- β — параметр, управляющий влиянием эвристики. Большее значение β означает большее влияние расстояния (1.0)
- J_i — множество всех возможных переходов из вершины i.

Обновление феромонов:

После того как все муравьи построили свои решения, количество феромонов на ребрах обновляется. Это происходит в два этапа:

- **Испарение:** Количество феромонов на всех ребрах уменьшается на некоторую величину ρ (коэффициент испарения равный 0.2):

  $\tau_{ij}(t+1) = (1 - \rho) * \tau_{ij}(t)$

- **Добавление феромонов:** Каждый муравей добавляет феромоны на ребра, которые он использовал в своем решении. Количество добавляемых феромонов зависит от качества решения (например, обратной длины пути

для задачи коммивояжера):

$$\tau_{ij}(t+1) = \tau_{ij}(t+1) + \Sigma_{k} \Delta\tau_{ij}^k$$

где:

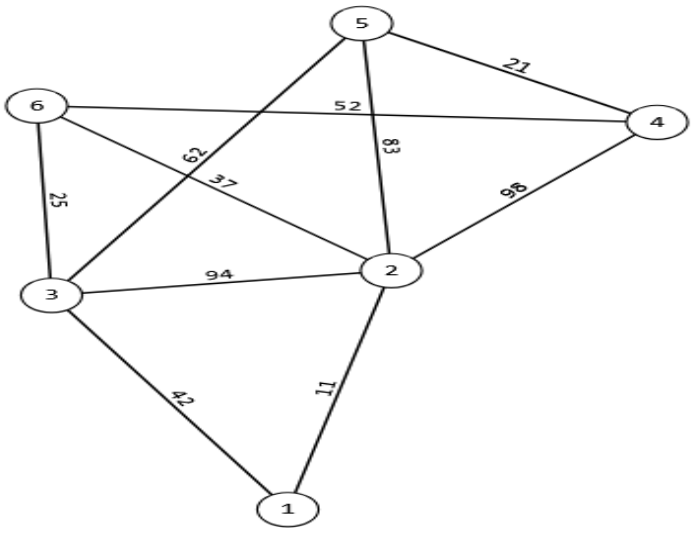
$$\Delta\tau_{ij}^k = Q / L^k$$

- Q — константа. (100)
- L^k — длина пути (стоимость решения), найденного муравьем k.

3. Практическая часть.

3.1 Демонстрация работы алгоритмов

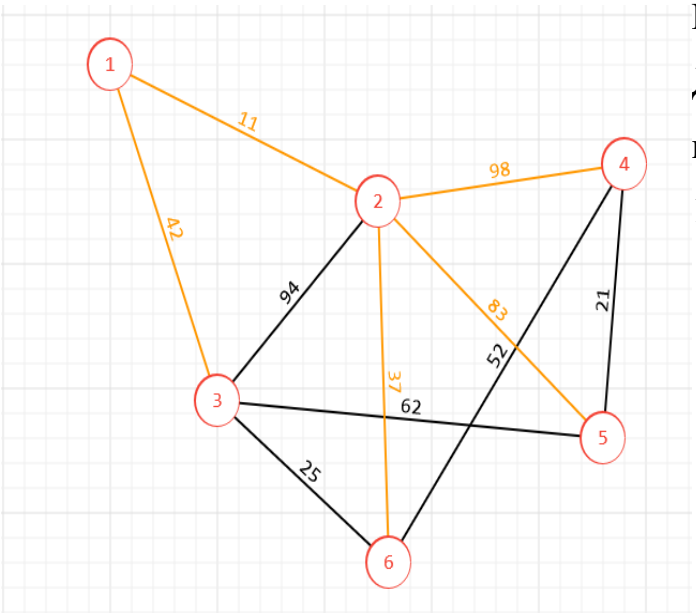
Перейдём к практической части исследования. Для начала был сконструирован тестовый взвешенный неориентированный граф и была написана к нему матрицу смежности.



Матрица смежности:

$$\begin{pmatrix} 0 & 11 & 42 & 0 & 0 & 0 \\ 11 & 0 & 94 & 98 & 83 & 37 \\ 42 & 94 & 0 & 0 & 62 & 25 \\ 0 & 98 & 0 & 0 & 21 & 52 \\ 0 & 83 & 62 & 21 & 0 & 0 \\ 0 & 37 & 25 & 52 & 0 & 0 \end{pmatrix}$$

Все последующие действия и алгоритмы в первой части моего проекта будут продемонстрированы на данном графе. Первым запустим BFS с началом в вершине 1:



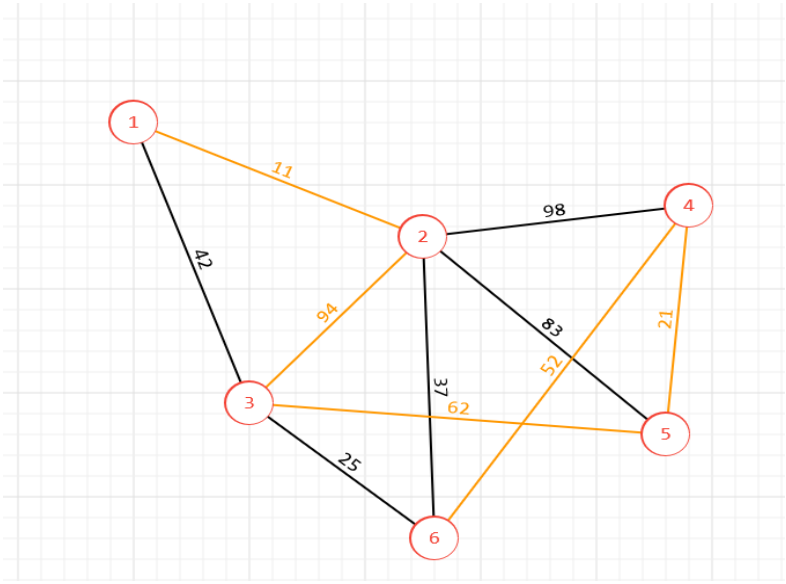
Мы получаем такой обход:

Для начала мы находимся в вершине 1, затем идем по маршруту 1->2->3->4->5->6

```
1 2 3 4 5 6
```

А при запуске DFS с началом в вершине 1 мы получаем:

По началу находимся в вершине 1, затем следуем маршруту 1->3->5->4->6->2




```
1 3 5 4 6 2
```

Далее на этом же графе запустим алгоритм Флойда — Уоршелла: Возьмем стартовую вершину 1 и найдем кратчайшие расстояние до вершины 3, запустив алгоритм, получаем такие выходные данные:

```
0 11 42 100 94 48
11 0 53 89 83 37
42 53 0 77 62 25
100 89 77 0 21 52
94 83 62 21 0 73
48 37 25 52 73 0
Short_path
4 6 2
```

Где сперва представлена матрица расстояний после работы алгоритма. А уже потом выведен кратчайший путь. Алгоритм корректно определил, что путь 2→6→4 (89) короче прямого ребра 2→4 (98), соответственно алгоритм работает корректно.

3.2 Решение практической задачи и анализ полученных данных

Для проведения экспериментального исследования я разработал программу, которая генерирует случайные взвешенные графы с заданным количеством вершин и применяет к ним три алгоритма решения задачи коммивояжера: полный перебор (Brute Force), жадный алгоритм (Greedy Algorithm) и муравьиный алгоритм (Ant Colony Optimization).

Методика эксперимента:

- Количество вершин варьировалось от 3 до 200.
- Для каждого значения N проводилось 10 запусков.
- Измерялось время выполнения (в наносекундах) и длина найденного маршрута.
- Результаты усреднялись.

Анализ таблицы результатов (Таблица 1):

Таблица содержит следующие столбцы:

	BF		GA		ACO		BF vs ACO (t%, path%)		BF vs GA (t%, path%)		ACO vs GA (t%, path%)	
nodes	time	path	time	path	time	path	time	path	time	path	time (10** 3)	path
3	14	14	3	14	667	14	2,10	100,00	466,67	100,00	22,23	100,00
6	40	18	5	18	1248	18	3,21	100,00	800,00	100,00	24,96	100,00
9	13153	27	11	34	4128	27	318,63	100,00	119572,73	79,41	37,53	125,93
12	18775966	28	14	30	8698	30	215865,33	93,33	134114042,86	93,33	62,13	100,00
20	-	-	29	46	35889	43					123,76	106,98
30	-	-	205	39	124405	50					60,69	78,00
50	-	-	61	81	485124	78					795,29	103,85
75	-	-	113	95	2118649	111					1874,91	85,59
100	-	-	445	117	4720935	140					1060,88	83,57
200	-	-	1316	216	10258577	301					779,53	71,76

nodes — количество вершин в графе.

BF, GA, ACO — время выполнения (time) и длина пути (path) для каждого алгоритма.

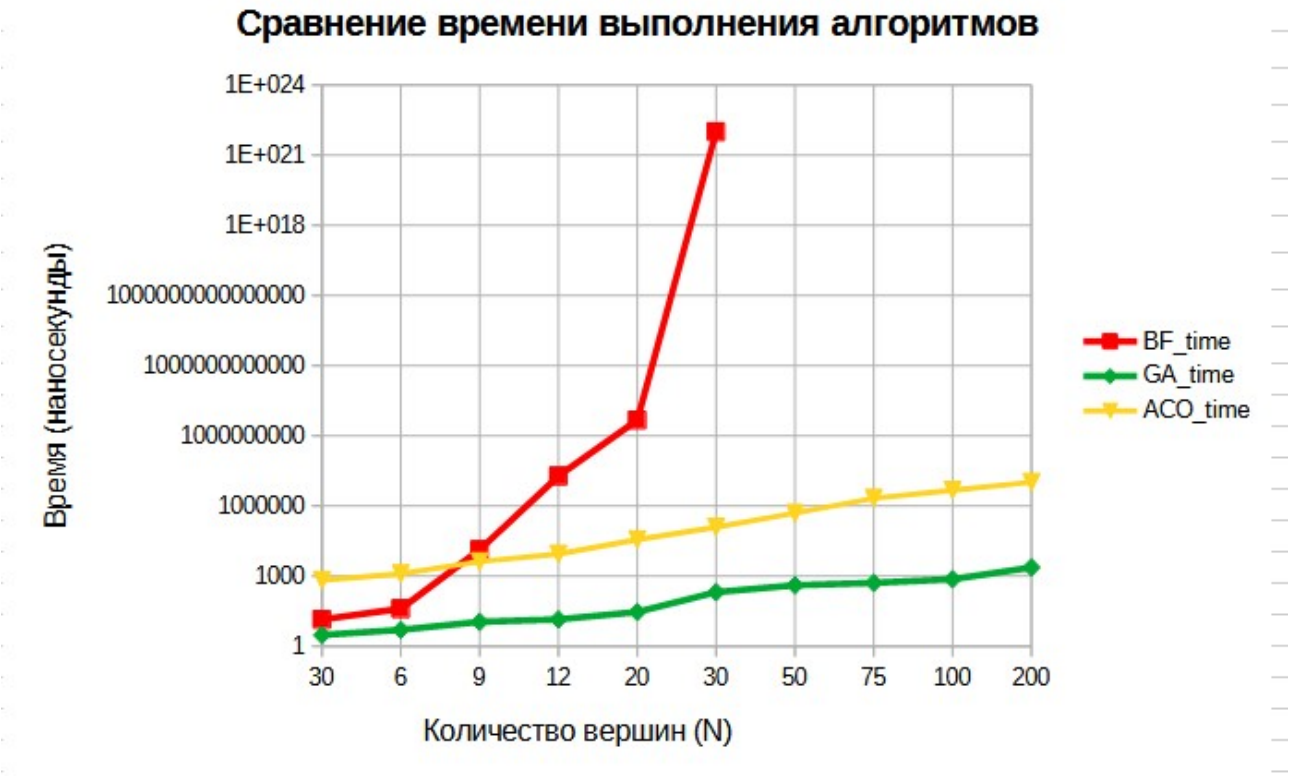
BF vs ACO, BF vs GA — процентное отклонение ACO и GA от полного перебора, принятого за эталон.

ACO vs GA — сравнение жадного алгоритма относительно муравьиного. Прочерки в таблице означают, что алгоритм не завершил работу за разумное время (более 2-3 часов).

1. Полный перебор (BF): Сложность O(n!) делает полный перебор непригодным для практического использования уже при N>12, несмотря на гарантированную оптимальность результата.

2. Жадный алгоритм (GA): Демонстрирует лучшую производительность ($O(n^2)$), но жертвует точностью, находя локально-оптимальные решения.

3. Муравьиный алгоритм (ACO): Обеспечивает баланс между точностью и производительностью, находя решения, близкие к оптимальным, за приемлемое время.



На графике представлена зависимость времени выполнения от количествавершин (N) для трех алгоритмов. Ось Y отображена в логарифмическом масштабе для наглядности.

Таким образом, ACO демонстрирует приемлемую масштабируемость, уступая жадному алгоритму в скорости, но значительно превосходя полный перебор.

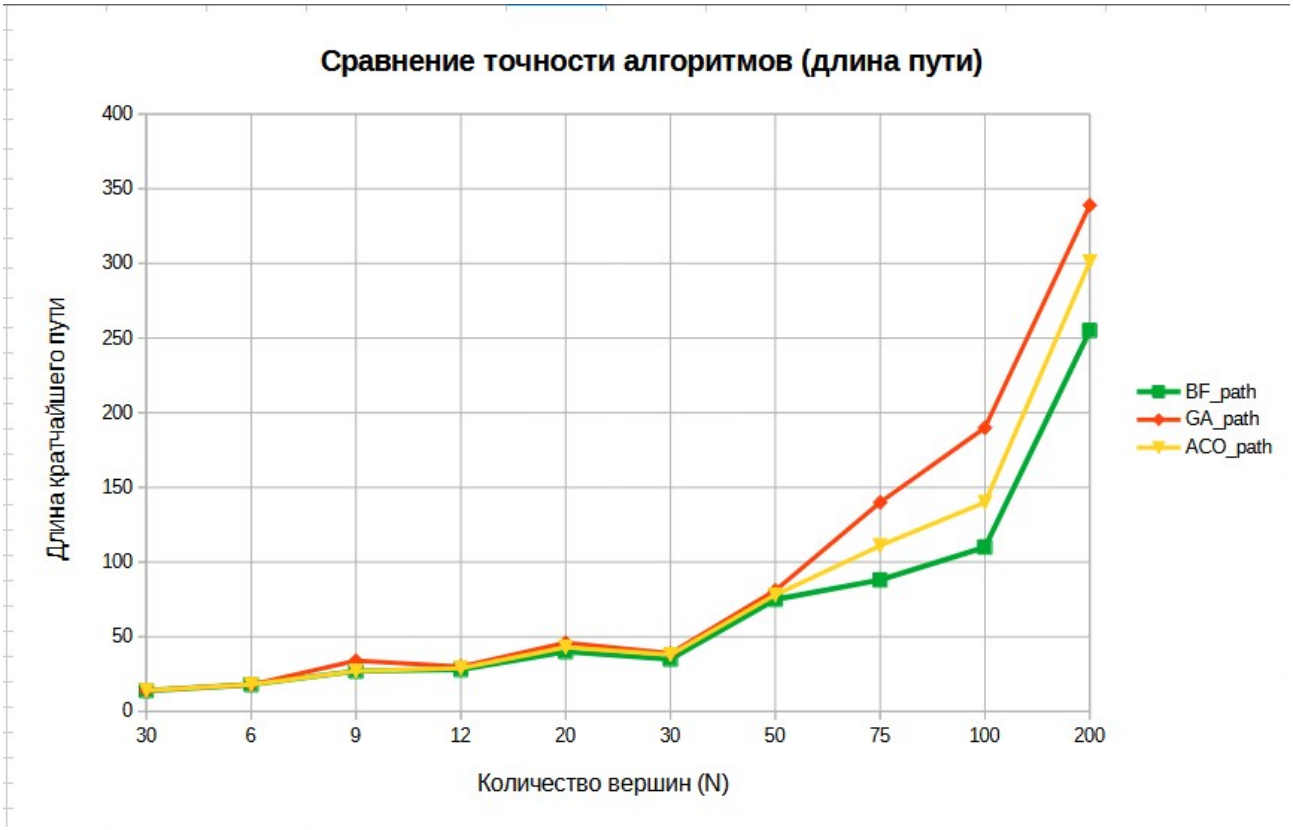


График показывает зависимость длины найденного маршрута от количества вершин.

Таким образом, ACO демонстрирует способность находить решения, превосходящие жадный алгоритм по качеству, особенно на больших графах.

Итоговый вывод по эксперименту:

Проведенное исследование показало, что:

1. Полный перебор применим только для малых графов ($N \leq 12$) из-за факториальной сложности $O(n!)$, но гарантирует нахождение оптимального решения.

2. Жадный алгоритм обеспечивает лучшую производительность $O(n^2)$, но находит решения с отклонением 20-30% от оптимального, что неприемлемо для задач высокой точности.

3. Муравьиный алгоритм (ACO) представляет собой оптимальный вариант, так как находит решения на 20-30% точнее жадного алгоритма, работает за приемлемое время (полиномиальная сложность), масштабируется до сотен или тысяч вершин.

4. Заключение

В заключение моей научно-исследовательской работы, посвященной изучению графов, отмечу выполнение задач, поставленных для достижени всех выдвинутых мною целей. В ходе проекта были разработаны и исследованы различные методы представления и хранения графа, способы его обхода, а так же алгоритмы для решения различных практических задач. Полученные результаты демонстрируют области применения графов и работу с ними. В частности, разработаны новые алгоритмы поиска кратчайшего пути, предложены решения практической задачи(задачи коммивояжера).

Практическая значимость проведенного исследования заключается в сравнительном анализе алгоритмов решения задачи коммивояжера, демонстрация алгоритма Флойда-Уоршелла и обходов графа (BFS и DFS). Разработанные в рамках проекта методы и алгоритмы могут быть применены в логистике, биоинформатике и в учебных целях.

Несмотря на достигнутые успехи, исследование выявило ряд перспективных направлений для дальнейшей работы. В частности, визуализация моей работы или перевод в приложение, корректировка для более простой работы с графами и разработка новых алгоритмов. Полученные результаты могут быть использованы для дальнейшего развития данной области знаний.

Список использованных источников и литературы

I Источники

1. https://ru.wikipedia.org/wiki Теория Графов
2. https://clck.ru/39kVSq
3. https://un-sci.com/ru/2019/06/20/istoriya-vozniknoveniya-teorii-grafov/
4. https://cdk-tsimlyansk.ru/presentations/teoriya-grafov-osobennosti-primeneniya-teorii-grafov-pri.html
5. https://habr.com/ru/companies/otus/articles/675730/
6. clck.ru/3Ew7z2

II Литература

1. Берж К. «Теория графов и её применения»
2. Харари Ф. «Теория графов»
3. Кристофидес Н. «Теория графов: алгоритмический подход»

Приложение

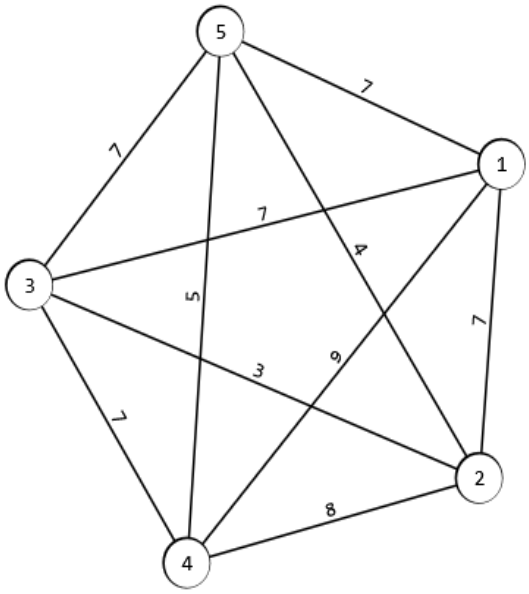
Протестируем на нескольких примерах работу моих алгоритмов:

```
Enter the number of nodes
5
Enter the start node
3
matrix:
0 7 7 9 7
7 0 3 8 4
7 3 0 7 7
9 8 7 0 5
7 4 7 5 0

BF:
solve path: 3 0 2 1 4 3
length: 28
time in nanosecond(1*10**-9second): 115

GA:
solve path: 3 4 1 2 0 3
length: 28
time in nanosecond(1*10**-9second): 0

ACO:
solve path: 3 4 1 2 0 3
length: 28
time in nanosecond(1*10**-9second): 1
```



Для данного примера я взял граф, состоящий из 5 вершин и указанным весом ребер, начал с 3 вершины. Для данного случая 3 алгоритма отработали одинаково хорошо, но время выполнения у них различается.

Далее рассмотрим еще один пример:

```
2 0 2 8 1 7 1 2 2 6
9 2 0 7 3 7 7 7 5 3
1 8 7 0 5 7 8 4 2 6
8 1 3 5 0 6 7 6 6 3
5 7 7 7 6 0 5 9 5 2
1 1 7 8 7 5 0 8 9 4
5 2 7 4 6 9 8 0 6 3
9 2 5 2 6 5 9 6 0 8
3 6 3 6 3 2 4 3 8 0

BF:
solve path: 8 2 4 1 7 9 5 6 0 3 8
length: 25
time in nanosecond(1*10**-9second): 3556

GA:
solve path: 8 1 4 2 9 5 0 3 7 6 8
length: 38
time in nanosecond(1*10**-9second): 0

ACO:
solve path: 8 3 0 6 1 7 9 5 4 2 8
length: 26
time in nanosecond(1*10**-9second): 6
```

